

ANALISI DI EFFICACIA E ROBUSTEZZA

Valutazione critica dell'infrastruttura Sovereign Quant Engine

Progetto: Sovereign Quant Engine (Jesse Multi-Agent Framework)

Analista: Antigravity AI Coding Assistant

Ambito: Controllo del Rischio, Automazione e Sicurezza Finanziaria

Data di Valutazione: Giugno 2026

Documento ad uso interno contenente l'audit strutturale e le metriche di affidabilità.

1. Sintesi della Metodologia del Progetto

Il Sovereign Quant Engine propone una soluzione robusta ad uno dei problemi più critici del trading algoritmico gestito da intelligenze artificiali: **le allucinazioni del codice e la violazione dei limiti di rischio**. Molti sistemi falliscono perché il bot di trading scrive la strategia ed esegue l'ordine direttamente, senza passaggi intermedi di convalida del rischio.

L'efficacia del Sovereign Quant Engine si basa su una **pipeline a ciclo chiuso divisa in tre fasi distinte**:

Fase dell'Engine	Meccanismo di Efficacia	Grado di Sicurezza
1. Supervisor & Contratti	Valida i file JSON degli agenti Alpha, Risk e Context prima che venga scritta qualsiasi riga di codice. Applica il <i>Bias di Rovina</i> bloccando all'istante drawdown > 2.0%.	Elevatissimo (Blocco statico)
2. Developer & Closed-Loop	Traduce la specifica in codice Python. Esegue il backtest e, in caso di errori di sintassi o import, corregge autonomamente il codice e ritenta l'esecuzione.	Alto (Autocorrettivo)
3. Stress Test Monte Carlo	Shuffla i rendimenti dei singoli scambi su 1.000 simulazioni basandosi su Win Rate e Profit Factor reali. Stima matematicamente la probabilità di rovina.	Elevato (Statistico)

Punto di Forza Chiave: Il sistema non si limita a dire 'il backtest ha funzionato', ma risponde alla domanda: 'Qual è la probabilità che, a causa della varianza casuale dei mercati, questa strategia superi il drawdown massimo?' Questo riduce drasticamente il rischio di overfitting.

2. Analisi dei Punti di Forza (Strengths)

L'architettura attuale presenta diversi elementi d'eccellenza dal punto di vista dell'ingegneria del software applicata alla finanza:

- **Cross-Validation Rigida:** L'aggiunta del controllo di coerenza sul rapporto rischio/rendimento minimo impedisce l'esecuzione di strategie con impostazioni logiche palesemente errate (es. stop loss troppo larghi rispetto al target).
- **Monte Carlo Dinamico e Coerente:** L'utilizzo del Win Rate reale estratto dal backtest, combinato con la formula corretta per ricavare il guadagno medio mantenendo fermo il Profit Factor, fornisce simulazioni di equity realistiche.
- **Generazione di Codice Jesse Nativo:** Invece di limitarsi a inserire commenti, il Developer Bridge genera properties Python effettive per gli indicatori tecnici e traduce le regole in codice booleano. La strategia generata è immediatamente importabile ed eseguibile all'interno di Jesse.

3. Aree di Vulnerabilità e Limitazioni (Weaknesses)

Nonostante l'elevata qualità dell'infrastruttura, sono state identificate alcune aree di miglioramento critiche per la produzione reale:

Vulnerabilità Identificata	Impatto Operativo	Mitigazione Consigliata
Modello Monte Carlo Binario	La simulazione assume che i trade siano solo vittorie (tutte uguali) o perdite (tutte uguali). Nella realtà, i profitti e le perdite seguono una distribuzione continua (Fat-Tailed).	Campionare direttamente dall'array storico dei singoli trade del backtest (Bootstrapping non parametrico).
Robustezza del Parser Logico	Il traduttore di espressioni in <code>`developer_bridge.py`</code> usa espressioni regolari semplici. Condizioni molto complesse o annidate potrebbero generare codice non valido.	Integrare un AST Parser (Abstract Syntax Tree) per convalidare e compilare le espressioni in modo formale.
Assenza di Gestione Regime Dinamico	Sebbene <code>`context_regime.json`</code> sia validato, i parametri della strategia (<code>params.py</code>) non cambiano dinamicamente in base allo stato del mercato (es. trend vs range).	Implementare selettori di parametri condizionali nel file <code>params.py</code> generato (regime-switching).

4. Roadmap Consigliata per la Produzione Reale (IRL)

Per portare il Sovereign Quant Engine ad un livello di livello istituzionale (pronto per il live trading reale con capitale significativo), si consiglia di seguire i seguenti passi di sviluppo:

Fase A: Ottimizzazione del Simulatore Monte Carlo

Implementare il campionamento statistico reale (Bootstrapping) sui trade eseguiti da Jesse. Invece di generare trade fittizi basati su Win Rate medio, lo stress test deve campionare casualmente (con ripetizione) dall'elenco effettivo dei trade storici. Questo catturerà automaticamente gli scostamenti insoliti e le sequenze di perdite consecutive reali.

Fase B: AST Parser per le Condizioni Alpha

Sostituire il parser a base regex in ``developer_bridge.py`` con una classe di compilazione basata sulla libreria standard ``ast`` di Python. Questo permetterà all'utente di scrivere formule complesse nelle specifiche (es. ``(rsi < 30 and close > sma) or macd_hist > 0``) garantendo che il codice generato sia sempre privo di errori di sintassi.

Fase C: Integrazione Live con Notifiche Telegram e Discord

Sviluppare un microservizio in Docker che monitora lo stato del bot live su Hetzner. Se il bot genera un errore o il drawdown in tempo reale si avvicina al limite del 2%, il servizio deve poter arrestare automaticamente il container Docker e notificare istantaneamente l'operatore tramite un bot Telegram.

Verdetto di Efficacia Globale: Il Sovereign Quant Engine è un'infrastruttura altamente efficace per il controllo del rischio. L'approccio rigoroso guidato da contratti JSON e convalidato da stress test statistici pone questo progetto ben al di sopra delle soluzioni di trading automatizzate retail, fornendo una reale barriera protettiva a difesa del capitale investito.